本單元名詞｜考場直接查

Explain / Define：寫中欄英文；例子看右欄。**How**：照 p.2 流程。**Why CFG / AST**：詳解 p.3 / p.4。本頁為 Ch3 術語與前兩次小考補充。

A｜字元、token 與文法

Ch3 PPT 2–5、31、51；Ch2＋補充

名詞 / notation	English definition（可作短答）	中文辨析 / 例子
Scanner / lexer	Groups source characters into tokens.	掃描器：字元流 → token 流。
Token / token type	A lexical unit / its category used by the parser.	type 如 ID、NUM；不是字元數。
Lexeme	The actual source character sequence matched.	詞素：ID 的 lexeme 可為 count。
Pattern / regular expression	Pattern: valid lexemes. RE: a description of a regular language.	RE 是規格；lexeme 是實例。
Attribute / transducer	Attribute: token data. Transducer: a recognizer that also produces output.	如 007 的數值 7；不只判合法。
Parser / CFG	Parser: checks token structure. CFG: each rule has one nonterminal on its left.	CFG 左側是一個 nonterminal。
Terminal / nonterminal	Terminals are not rewritten; nonterminals are expanded by productions.	parser 文法的 terminal 通常是 token type。
Production / start symbol	A rewrite rule / the symbol from which derivation begins.	如 S→aSb；從 S 開始推導。
AST / parse tree	An AST keeps essential structure; a parse tree records grammar expansions.	AST 留運算 / 敘述，省語法包裝。

B｜語言集合與 RE operators

PPT 5–16

名詞 / notation	English definition（可作短答）	中文辨析 / 例子
Alphabet Σ / string / language	A finite symbol set / a finite sequence / a set of strings.	符號集合 / 一串符號 / 字串集合。
Empty string λ or ε / ∅	λ is a zero-length string; ∅ contains no strings.	{λ}≠∅；λ 不消耗輸入。
Regular language	A language recognizable by a finite automaton.	RE、NFA、DFA 表達同類語言。
Literal / metacharacter	A literal denotes itself; a metacharacter controls the pattern.	字面 a；* 是 RE 操作符。
Alternation A | B	The union of the two languages.	選 A 或 B；不是串接。
Concatenation AB	Strings from A followed by strings from B.	順序重要；通常 AB≠BA。
Kleene A* / positive A+	Zero-or-more / one-or-more concatenated repetitions.	* 一定含 λ；A 含 λ 時 + 也可含 λ。
Optional A? / A^k	Zero or one occurrence / exactly k concatenated copies.	A?=A|λ；A^0=λ。
Character class / Not(C)	A set of single characters / its complement within Σ.	Not(C)=Σ\C；非任意語言補集。

C｜Finite automata 與子集構造

PPT 23–50

名詞 / notation	English definition（可作短答）	中文辨析 / 例子
DFA / NFA	DFA: one successor per symbol. NFA: multiple choices or epsilon moves.	能力相同；NFA 接受看存在路徑。
State Q / q_0 / F	Q is the finite state set; q_0 starts; F accepts.	M=(Q,Σ,δ,q_0,F)；雙圈表示 F。
Transition δ / Δ	A DFA successor / a set of NFA successors.	轉移表：row=state，col=輸入。
Epsilon transition	A move that consumes no input character.	λ 邊不代表讀一個 λ 字元。
λ-closure close(S)	States reachable from S by zero or more epsilon moves.	包含 S 本身；沿 λ 走到固定點。
move(S,c)	All successors after consuming one c from states in S.	先聯集，之後才做 close。
Subset construction	Uses sets of NFA states as DFA states.	集合 S 含任一接受 state 即接受。
Dead state / sink	Dead: no acceptance is reachable. Sink: every input loops there.	本課 X=∅ 是拒絕 sink。
Worklist / seen	Unprocessed discovered subsets / all discovered subsets.	新集合只入隊一次；按內容去重。

D｜掃描行為與錯誤

教材整理＋補充辨析

名詞 / notation	English definition（可作短答）	中文辨析 / 例子
Maximal munch / tie priority	Take the longest match; flex breaks equal-length ties by rule order.	floatx 選整個 ID；同長才比先後。
Lookahead / backtracking	Inspect upcoming input / return to a saved input position.	PEEK 不吃；ADVANCE 吃；記最後接受。
Recognize / tokenize	Test a whole string / split input into successive tokens.	完整 NUM 拒 12e；仍可能切 NUM+ID。
EOF / EOL	End of input / end of line.	檔尾≠換行；教材 // 註解要求 EOL。
Semantic action / semantic analysis	An action attached to a rule / checks of declarations and types.	本課 action 建 AST；型別另檢查。

三個判定句：Lexical error: no valid token match. Syntax error: the token sequence violates the grammar. Static semantic error: valid syntax violates declaration or type rules.　**注意**：接受整串必須讀完；scanner 切 token 則取合法前綴。

常見流程｜看到題目就照步驟做

直接定位：Define / Difference → p.1；scanner 如何建立 → F1；切 token → F2；RE→NFA → F3；NFA→DFA → F4；畫 AST → F5；判 error / Why CFG → F6。本頁為課程流程摘要與補充微型例題。

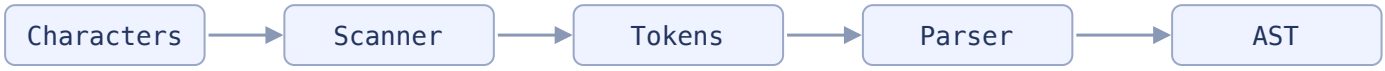
F1｜建 scanner 與跑 compiler：兩條流程

PPT 2、33；Ch2＋補充串連

建立時：用 token patterns 建 NFA，子集構造成 DFA，再輸出表或控制程式。



執行時：scanner 依 pattern 切字元；parser 依 CFG 組 token，建 AST。



後續：AST ＋ symbol table → 檢查宣告 / 型別 → decorated AST → 產碼。圖為本課主線摘要；**RE→NFA 是建立工具，characters→tokens 是處理來源程式。**

F2｜給短字串，列 tokens

詳 p.6

① 看題目 token 規格 / 語言。② 從當前位置找最長合法前綴。③ flex 同長取先列 rule。④ 回 type，附必要 attribute；略過空白 / 註解。⑤ 移到下一 token，直到 EOF。

例：keyword=float，ID=本章 identifier：
`float x=12;` → FLOAT、ID(x)、ASSIGN、NUM(12)、SEMICOLON。`floatx` → 一個 ID。

長候選失敗：回最後接受位置；沒有接受前綴才無 token 可回。
Choose the longest valid prefix; resolve equal-length matches by rule priority.

F3｜給 RE，畫 NFA

模板 / 完整例 p.7

① 先拆 (…)、* / +、串接、|。② literal 畫一條字元邊。③ | 加 λ 分支；AB 接 A 出口到 B 入口。④ * 加零次旁路及重複邊；+ 去掉外部零次旁路。⑤ 標整體 start、F、Σ。

`(bc)*d`：先 b 串 c → 對 bc 加 * → 再串 d。測 **d ✓、bcd ✓、λ ✗、bc ✗**。

交卷檢查：所有邊有 label；只把完整表達式出口標接受。若內層 A 含 λ，A+ 仍可收 λ。

F5｜給短程式，畫 AST

4 棵完整樹 p.4

① 確認語言及結合性。② 按來源順序列敘述。③ 指定句先畫 assign（目標，RHS）。④ RHS 按括號 / 優先序 / 結合性遞迴拆。⑤ 不畫純語法包裝，保留識別字與常數。

一般 C-like `x=a+b*c;`：
`assign(x, plus(a, times(b,c)))`。
整句根=assign，RHS 根=plus，不可把 + 畫在 * 的下面。

以還原括號式驗算。題目要 AST，就畫節點與邊；未要求時不先常數摺疊。**教材 ac 的右結合差異見 p.4。**

F4｜給 NFA，轉 DFA

完整演算 p.8

① 起始 A=close({q_0})。② 對 A 的每種 c：聯集 move(A,c)，再 close。③ 新集合命名並入 worklist；相同集合沿用名稱。④ 重複直到 worklist 空。⑤ 含任一 NFA 接受 state 的集合標接受；補 ∅ sink。

$$\delta_D(S,c)=\text{close}(\text{move}(S,c))$$
$$S \text{ 接受} \Leftrightarrow S\cap F_N\neq\varnothing$$

微型題（自編）：Q={1,2,3}、Σ={a}，start=1、F={3}；只有 1—λ→2、2—a→3，其餘無邊。

A=close({1})={1,2}；move(A,a)={3}，再 close 得 B={3}；B 讀 a 得 X=∅。

DFA / NFA 集合	a	接受？
→A={1,2}	B	否
★B={3}	X	是
X=∅	X	否

a ✓：A→B。**λ ✗：**停 A。**aa ✗：**A→B→X。判整串要讀完；NFA 是**存在**成功路徑。Move on one symbol, take epsilon closure, and reuse equal subsets.

閉包可走零條 λ 邊，包含起點；∅ 若不可達就不列進可達集合。最小化：先分接受 / 拒絕，再按後繼組別反覆拆；補充例 p.9。

F6｜判錯誤 / 解釋 Why CFG

詳 p.3

先逐層判：字元能切 token？→ token 符合 grammar？→ 宣告 / 型別符合規則？判定需依題目所給語言。

C-like 假設例	第一個問題
@ 無合法 token pattern	Lexical
x = ; 缺 RHS	Syntax
x 未宣告但 x=1 文法合法	Static semantic

The input violates the token pattern / grammar / declaration or type rules. 作答要補**具體原因**，不要只寫 error 名稱。

Why CFG：任意深度巢狀 → RE / 有限狀態不夠 → CFG 可遞迴描述 → 實用 parser；宣告與型別交 semantic analysis。
CFGs describe recursive, nested syntax beyond regular languages and support practical parsing.

交卷前 10 秒：token type≠lexeme；λ≠∅；字元≠token；NFA state≠DFA 子集；closure 不吃字元；接受完整字串要讀完；AST 的左右 / 敘述順序要保留。**看不懂題目名詞先查 p.1，數學構造直接照 F3 / F4。**

Chapter 3 | Scanning 考場速查

找答案：名詞 → p.1；流程 → p.2；CFG → p.3；AST → p.4；RE → p.5；掃描 → p.6；RE→NFA → p.7；子集構造 → p.8；abb / 最小化 → p.9；綜合題 → p.10。
題型依據：第一次：Explain、Difference、情境判斷；第二次：Why CFG、短程式 AST。以下是**複習用作答範本與自編練習**，不代表本次考題或教師標準答案。

1 | Why use a context-free grammar instead of others?

Quiz 2；Ch3 PPT 51–52＋補充

可直接寫：Context-free grammars can describe recursive and nested syntax, such as balanced parentheses, which regular grammars cannot express in general. They offer a useful balance between expressive power and practical parsing. More powerful grammar classes make general parsing harder, while declaration and type constraints are usually checked separately during semantic analysis.

中文抓分：① 足以表達無上限巢狀 / 遞迴結構；② 比 regular grammar / RE 強；③ 比更強文法容易設計實用 parser；④ 語法與型別 / 宣告檢查分工。**不要只寫「CFG 比較快」**，RE 的 DFA 掃描通常反而很有效率。

規格	擅長什麼	作答時的界限
RE / regular grammar	identifier、number、keyword 等 token 形狀	有限狀態；不能一般性地配對任意深度括號
CFG	expression、statement、program 的遞迴結構	A → α：左側只有一個 nonterminal；不保證無歧義或 LL(1)
Context-sensitive / unrestricted grammar	可描述更廣的字串限制	一般剖析更複雜；日常 compiler 多將相關限制交 semantic analysis

最短版：CFGs express nested program structure beyond regular languages and support practical parsing; semantic analysis handles declaration and type constraints.

2 | Scanner / Parser / Semantics

階段	輸入 → 輸出 / 任務
Scanner / lexer	characters → tokens；RE 辨認詞素
Parser	tokens → syntax structure / AST；依 CFG
Semantic analysis	AST ＋ symbol table；宣告 / 型別檢查
Code generation	checked representation → target code

The scanner groups characters into tokens. The parser checks token structure using a CFG and builds a syntax representation. Semantic analysis checks meaning-related constraints such as declarations and types.

建 scanner：RE → NFA → DFA → table / code。
跑 scanner：source characters → tokens。
兩條流程不可倒混；scanner **不負責畫 AST**。

3 | Token / Lexeme / Attribute

PPT 3、31

A token has a type and, when needed, an attribute. The lexeme is the actual source character sequence matched by the token pattern.

`count`：type=ID，lexeme="count"，attribute 可為符號表索引。
`007`：lexeme="007"，數值 attribute 可為 7。

Pattern 是規格；**lexeme** 是實際字串；**token type** 是 parser 用來分類的名稱。不同 lexemes 可屬同一 type。

4 | CFG 強在哪？一個可寫的例子

令 $L = \{ a^n b^n \mid n \geq 0 \}$，CFG：

$$S \rightarrow aSb \mid \lambda$$

`S ⇒ aSb ⇒ aaSbb ⇒ aaaSbbb ⇒ aaabbb`。每次多一個 a，同時多一個 b；λ 結束，a 都在 b 之前。

有限 DFA 反證：若有 m 個狀態，讀 a^0、a^1、…、a^m 後至少兩個字首 a^i、a^j（i<j）到同一狀態。再接 b^i，DFA 必須同判；但 $a^i b^i$ 應收、$a^j b^i$ 應拒，矛盾。

> **不要寫：**「aaabbb 不是 regular」。單一字串可直接用 RE `aaabbb`。不能表達的是**任意 n 的整個等量語言**。

括號例：S → (S)S | λ 可表達平衡括號。CFG 的「context-free」指改寫 A 不需看左右上下文，**不是程式不用考慮 scope**。

5 | 情境題：先說哪一層失敗

現象（依指定語言規則）	階段
@ 無任何 token pattern	Lexical error
x = ;，每個 token 合法但缺 RHS	Syntax error
x 未宣告，但 x=1 文法合法	Static semantic error
文法 / 型別合法，結果違反人的意圖	可能是 logic error

A syntax error violates the grammar; a static semantic error violates declaration or type rules despite a grammatically valid token sequence.

AST｜看到短程式，怎麼畫？

6｜四步畫樹＋一個驗算

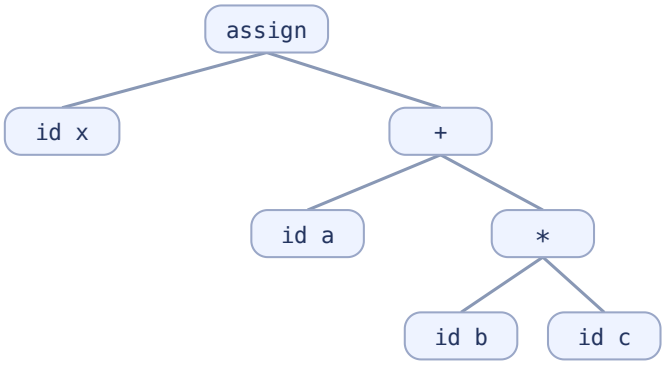
Ch2 PPT 27–28、39–42＋自編例題

① **分敘述：**program / block 依來源順序接 declaration、assignment、print 等。② **找最外層：**如 `x=E` 先畫 assign，左目標、右 RHS。③ **遞迴分解：**先按括號、再優先序、再結合性決定 expression 的根。④ **保留：**operators、識別字、常數、必要的敘述結構；省略純語法包裝與分隔符。⑤ **驗算：**由樹還原括號式，檢查左右、先後與題目相符。

An AST represents the essential syntactic structure using operators and operands. It omits grammar-only nodes and punctuation while preserving precedence, associativity, and statement order.

> **第二次小考未附實際短程式。**下方 A/B 是**一般 C-like 補充練習**，不是原小考答案；教材 ac 沒有乘法、括號或分號。先辨認題目採哪套語言。

7｜A：x = a + b * c;

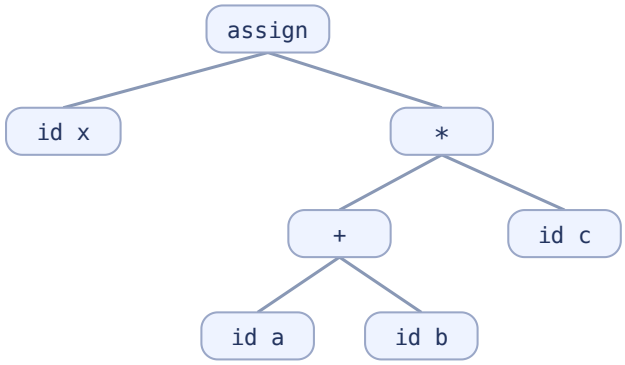


RHS 的根是 +：因 * 優先於 +，`b*c` 作一個子樹。整句的根仍是 assign。

```
assign(id(x),
       plus(id(a), times(id(b), id(c))))
```

上式是樹的文字記法；要求畫樹時，畫節點與邊。葉標 a 或 id(a) 須一致。

8｜B：x = (a + b) * c;



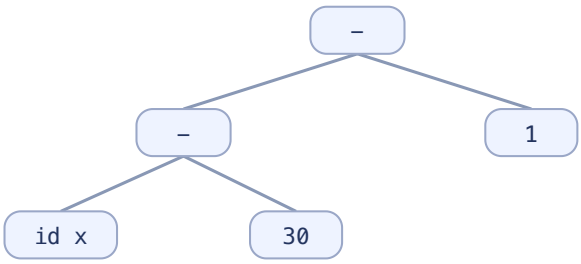
RHS 的根改成 *：括號強制先加；AST 通常不畫「(」「)」節點，**樹形本身保留分組**。

```
assign(id(x),
       times(plus(id(a), id(b)), id(c)))
```

Expr、Term、Factor、分號、λ 屬 parse tree 的語法細節。

9｜結合性：同優先序時怎麼拆？

一般 C 減法左結合：`x-30-1` → `(x-30)-1`；最外層是**最後一個 −**。



x=100 時左樹得 69；右樹 `x-(30-1)` 得 71。左右運算元不能交換。

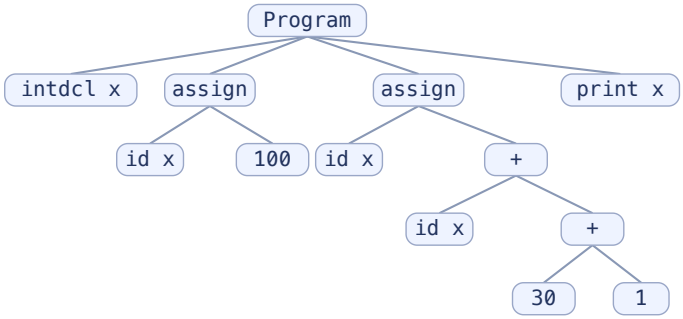
一般 C 指定右結合：`x=y=z` → `assign(x,assign(y,z))`。**一元負號** `-x` 畫 neg(x)，與 binary minus(L,R) 分開。

右遞迴文法不必強迫右結合 AST；semantic actions 可累積左子樹。以題目指定的語意 / 教材圖為準。

10｜教材 ac：原圖是右結合

Ch2 p.42

`i x  x=100  x=x+30+1  p x`



重繪 Ch2 p.42 的樹形；補明 declaration / token 名稱。

> **依教材圖：**plus(x,plus(30,1))。**一般左結合：**plus(plus(x,30),1)。本例整數值同為 131，但不能任意把右樹當所有語言的標準。

11｜容易失分：AST ≠ Parse Tree ≠ Decorated AST

表示	保留什麼	作答提醒
Parse tree	文法的 nonterminals、terminals、每次改寫	葉序要符合 token 序列；依要求畫 λ
AST	核心操作與 operands、敘述 / 宣告結構	保留所有敘述；不要只畫 RHS 或算出結果
Decorated AST	AST 加型別、符號資訊，可能插 int2float	只有要求型別分析時才加入；普通 AST 不先常數摺疊

Why use an AST? It removes grammar-specific detail and gives later analysis and code generation a compact representation of program structure.

Regular Expressions | 寫規格與判合法

12 | RE 是語言：一個字串集合

A regular expression specifies a regular language: the set of strings that match its pattern. A finite automaton recognizes that language.

Σ：有限字母表；**Σ***：Σ 上所有有限長字串。**λ / ε**：空字串，|λ|=0。**∅**：空集合，沒有任何字串。**L(λ)={λ} ≠ L(∅)=∅**；教材用 0 表空集合，不是 literal "0"。

形式	語意 / 展開	例與陷阱
A | B	L(A) ∪ L(B)；選任一支	a|bc 接受 a 或 bc，不接受 abc
AB	先 A 再 B；{xy | x∈L(A), y∈L(B)}	一般 AB ≠ BA；Aλ=λA=A
A*	0 次以上；λ | A | AA | …	一定含 λ；∅*={λ}
A+	1 次以上；AA*	若 A 自己含 λ，A+ 仍可能含 λ
A?	0 或 1 次；A | λ	(ab)? 接受 λ 或 ab
A^k	串接恰 k 次；A^0=λ	$(ab)^2$ 是 abab
Not(C)	字元集合 Σ\C，C⊆Σ	只排除單一字元集合；非任意語言補集

優先序：分組 (…) → postfix * + ? → 串接 → |。**Literal**：要匹配字面 *、+、.，依工具 syntax 跳脫或用字元類別。`[.]` 是字面點；多數 regex 工具的 `.` 是 wildcard，是否含換行取決於 dialect / 設定。

13 | 最常見的分組判斷

RE	整串可接受	整串不可接受
ab*	a, ab, abb, …	λ, b, abab
(ab)*	λ, ab, abab, …	a, abb
(abc+)+	abc, abcc, abcabc	λ, ab, abbc
(a|b)*abb	abb, aabb, babb, ababb	λ, ab, abba

不要看字元各自出現過就說合法。`(abc+)+` 每塊都是一個 a、一個 b、至少一個 c，然後整塊至少一次。

`A∅=∅A=∅`；`A|∅=A`。空字串是串接的單位元，空集合是會讓串接全空的集合。

14 | Identifier：本章特有規格 p.15、32

```
L = [A-Za-z]    D = [0-9]
L(L|D)*(_(L|D)+)*
```

讀法：字母開頭；後接任意字母 / 數字；每個底線後至少一個字母 / 數字。允許 `foo_2_bar`，拒 `_foo`、`foo_`、`foo__bar`。

底線不是任意可放；`[A-Za-z_][A-Za-z0-9_]*` 是**另一套**常見 ID 規格，不能代替本頁教材。

15 | 格式符合不代表語意正確

p.16 日期規格允許年 1900–2099、月 01–12、日 01–31，所以 `2001-02-31` 形狀符合，日曆不合法。兩個分隔符獨立，`2001/02-03` 也符合。

字串 `"[^"\r\n]*"` 禁 quote / CR / LF，但**沒定義跳脫引號**；不能當完整 C string 規格。

16 | 科學記號：兩層 optional HW 3；補充修正版

```
^[0-9]+([.][0-9]+)?
        (e[+-]?[0-9]+)?$
```

排版換行僅供閱讀；實際 pattern 不含換行或上述空格。

① **整個 exponent 可省略**：`(e…)?`。② **exponent 內符號可省略**：`[+-]?`。e 後數字始終**至少一位**。沿用 ac：小數點兩側必須有 digit。

測試字串	整串結果 / 理由
12, 3.2	✓ exponent 可省
1.0e10, 123e-22, 12e+3	✓ 無號 / 負號 / 正號
12e, 1.2e-	✗ 缺 exponent digits
.1, 10., 3.e2	✗ 小數兩側須 digit
1E2	✗ 這個規格只收小寫 e

p.54 原解答的缺漏：e 部分變必填；只寫 -? 漏正號；fnum 小數前少 +。問「依題意擴充」使用上方修正版；NUM 如何分 int / float 是另外的語意政策。

Make the entire exponent optional, allow an optional plus or minus sign, and require at least one exponent digit.

17 | Whole match / Search / Tokenize

整串驗證：要求所有字元屬同一 pattern，例 `^…$`（依工具模式）。**Search**：找一段子字串。**Scanner**：從目前位置取合法 token 前綴，然後繼續。

`12e` **不符合完整 NUM**；但某 scanner 可切 `NUM(12), ID(e)`。要把它報 malformed number，須另定規則。不要由整串 regex 拒絕直接斷言 lexer 必報錯。

Dialect 提醒：理論 RE 的 λ、Not 是數學記法；工具的 `\d`、dot、anchors、BRE / ERE 有自己的規則。上方數值採 POSIX ERE 形式；不用把所有語法混在同一答案。

Finite Automata × Scanner 行為

18 | 五元組與 DFA / NFA

PPT 23–36

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q：有限 states；**Σ**：輸入字母表；**δ**：轉移；q_0：起始 state；**F⊆Q**：接受 states。圖中 incoming arrow 是起始，雙圈是接受；一個 state 可同時是兩者。

比較	DFA	NFA（本章允許 λ 邊）
轉移	δ: Q×Σ → Q；每個 state / 字元唯一後繼	Δ: Q×(Σ∪{λ}) → $\mathcal{P}$(Q)；後繼是一個集合
λ / ε 邊	無；每一步吃一個字元	可有；移動不吃字元
接受完整 w	讀完所有字元後，當前 state 在 F	存在一條吃完整 w、可含 λ 的路徑到 F
表達能力	Regular languages	同 DFA；較方便組合，並沒有更強

A DFA has one next state for each state and input symbol. An NFA may have multiple choices and epsilon transitions. They recognize the same class of languages.

> NFA 不是要求所有路徑成功；一條成功即可。DFA 中途到接受 state 也不能忽略後面的字元。**∅ 集合**可成為 DFA 的 dead state，與空字串 λ 不同。

19 | Comment DFA：逐字追蹤

p.25–30

教材：`//(Not(EOL))*EOL`。OTHER 是非 /、非 EOL 字元；X 表拒絕 sink。EOF 是結束條件，**不是輸入字元 EOL**。

State	/	EOL	OTHER	結束時
→1	2	X	X	拒
2	3	X	X	拒
3	3	4	3	拒
★4	X	X	X	接受
X	X	X	X	拒

`//hi\n`：1→2→3→3→3→4 ✓。`//hi` 在 EOF 留在 3 ✗，因教材要求 EOL。這不是宣稱所有真實語言都拒絕檔尾註解。

Table-driven：資料表 δ + 通用 driver。**Explicit control**：if / switch / loops 直接編入轉移。**Transducer**：辨識之外還產出 token / attribute，例如 digit 串轉數值。

Table-driven scanning separates transition data from control code. Explicit control embeds transitions in branches and loops.

20 | 整串辨識器手算模板

設 q=q_0 → 依每個字元查下一 state → 不合法邊到 X → **字串讀完**才判 q∈F。表中沒畫的邊通常表示失敗；若要求完整 DFA，就補 sink。

字母表先寫清楚：本頁 comment 表用「字元類別」合併大量字元；小題 a/b/c 的表則按個別符號列。

21 | Maximal munch + Tie priority

p.20；flex 補充

Choose the longest matching lexeme. If multiple flex rules match the same length, choose the rule listed first.

keyword 與 ID 重疊：把 keyword rule 放 ID 前面。同長才比順序；**不能先看到 keyword 就停止**。

輸入	應選	理由
float	FLOAT keyword	與 ID 同長；keyword 規則先列
floatx	ID(floatx)	ID 6 字元長於 keyword 5
int7	ID(int7)	以本章多字元 ID 規格為例
5+10	NUM(5), PLUS, NUM(10)	每次從目前位置重新匹配

通用 scanner 需記最後接受：掃描時更新 `last_accept_state` 與 `last_accept_position`；遇到死路，回到最後接受處，回傳那個 token，剩下字元交下一輪。

> **反例**：只有 `a`、`abc` 兩種 token，輸入 `abx`。讀 a 已接受；讀 b 尚有可能；讀 x 失敗。應回退輸出 token `a`，從 b 繼續 / 報錯。**只檢查最後 state 會漏掉合法前綴**。

教材 p.28 的簡化 driver 只查停下時的 state，不能直接視為處理所有 token 規格的 maximal-munch implementation。

22 | 小 token 題：honeycone=5+10*num;

```
ID(honeycone), ASSIGN(=), NUM(5),
PLUS(+), NUM(10), TIMES(*), ID(num),
SEMICOLON(;)
```

共 8 tokens，與空白 / 字元個數不同。此例取自 p.3 的 C-like token 示意，**不是** ac 的單字母 identifier 文法。

略過 whitespace / comment 的 action 可繼續掃，不交 token 給 parser。`yylval` 傳必要 attribute；`return` 交 token type。

RE → NFA｜按結構拼，別猜箭頭

23｜先解析 RE，再組片段

PPT 37–41；補充操作順序

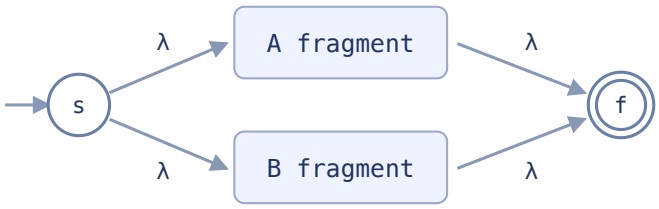
順序：① 先圈群組與 operators；② literal 用一條吃字元的邊；③ 按 |、串接、* 組合；④ 標一個整體 start 與 accept；⑤ 用 λ、最短合法字串、近似錯例驗算。狀態編號可不同，**結構與語言正確才重要**。

a：s —a→ f；**λ：**s —λ→ f；**∅：**沒有從 s 到 f 的路徑。所有子片段 state 先重新編號以免撞名；組合後移除子片段的「整體接受」標記，只保留完整表達式的出口。

24｜三個可套模板

依教材重繪

A｜B：分支取任一條



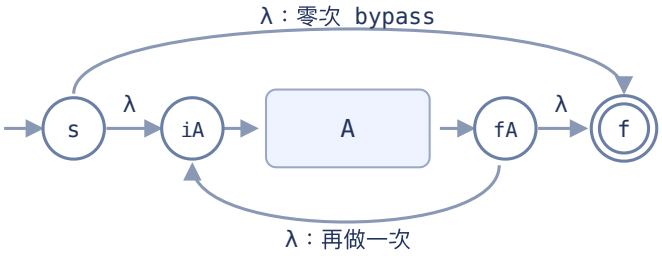
新起點 λ 分到 A / B；兩出口 λ 到新終點。只要一支完成即可接受。

AB：先 A，再 B



A 的出口 λ 接 B 的入口；**只讀完 A 不算完成 AB**。

A*：零次旁路＋重複迴圈



A+：保留重複邊，移除新 s→新 f 的零次旁路。**A?：**A｜λ；有旁路，沒有重複邊。

> 若 A 自己可接受 λ，去掉外部 bypass 後 A+ 仍可能接受 λ。例 (a*)+ 可收空字串；「+ 不收 λ」須先確認內層不收 λ。

25｜HW 5(d)：(bc)*d

PPT 55–57

語言：零個以上 bc，最後**恰一個 d**。λ 不收，但 d 收。教材 NFA：start=1，F={6}，Σ={b,c,d}。

NFA	b	c	d	λ
→1	∅	∅	∅	{2,5}
2	{3}	∅	∅	∅
3	∅	{4}	∅	∅
4	∅	∅	∅	{2,5}
5	∅	∅	{6}	∅
★6	∅	∅	∅	∅

分解：1→5 的 λ 允許零次 bc；2—b→3—c→4，4→2 的 λ 再做一次；4→5 的 λ 離開迴圈，再吃 d 到 6。

轉 DFA：先算集合（公式見 p.8）

A={1,2,5}；B={3}；C={2,4,5}；D={6}；X=∅。→ 是 start，★ 是接受。

DFA	b	c	d
→A	B	X	D
B	X	C	X
C	B	X	D
★D	X	X	X
X	X	X	X

一格怎麼算：A 在 b 下只有 2→3，所以 move={3}、close={3}=B。B 在 c 下到 4，再走 λ 到 2、5，得到 C。

bcd：A→B→C→D ✓。**bc：**A→B→C ✗。**bcdd：**A→B→C→D→X ✗；先到 D 不代表剩下 d 可忽略。

最小化補充：A、C 轉移完全相同且都拒絕，可合併；連同 B、D、X 得 4 states。先做完子集構造，再談合併。

26｜考試畫 NFA 的五個檢查點

Start / F / Σ 都有；每條邊明寫字元或 λ；* 的零次 / 重複都有；+ 不誤加外部零次邊；串接不讓前半完成就接受。

Construction answer: Build NFAs for the subexpressions, connect them according to alternation, concatenation, and closure, and mark only the final exit of the complete expression as accepting.

NFA → DFA｜集合就是一個狀態

27｜三個公式＋接受條件

PPT 42–48

$$S_0 = \text{close}(\{q_0\})$$
$$\text{move}(S,c) = \cup_{s\in S}\, \Delta(s,c)$$
$$\delta_D(S,c) = \text{close}(\text{move}(S,c))$$
$$S \text{ 是接受 state} \Leftrightarrow S \cap F_N \neq \varnothing$$

close(S) / λ–closure：從 S 本身開始，沿**零條或多條 λ 邊**到的全部 states。遇循環就去重，反覆加入到沒有新 state；不吃字元。
move(S,c)：所有成員吃**一個 c**的後繼聯集；之後一定再 close。

Each DFA state represents all possible NFA states after the same input prefix, including epsilon reachability. It is accepting if it contains at least one accepting NFA state.

28｜教材 Fig 3.24 的原 NFA

p.43

start=1，F={5}，Σ={a,b}；只有 1→2 是 λ。

NFA	a	b	λ
→1	{3}	{1}	{2}
2	{4,5}	∅	∅
3	∅	{4}	∅
4	{5}	{5}	∅
★5	∅	∅	∅

Step 1：起始閉包

{1} → 加入 2 → {1,2}，再無新 λ 後繼。令 A={1,2}。不含 5，故**空字串拒絕**。

> **p.47 原句把 3 寫入 closure，應校正。**1→3 是 a 邊；不讀 a 就不能把 3 加進起始閉包。

Step 2：算 A 的所有字元

```
move(A,a) = {3} ∪ {4,5}
          = {3,4,5}
close(...) = {3,4,5} = B

move(A,b) = {1} ∪ ∅ = {1}
close({1}) = {1,2} = A
```

B 含 5，標接受；A 已存在，不再造新 state。**同一集合就是同一 state**，排列順序不影響身分。

Step 3：算 B、其餘新集合

B 在 a 下只有 4→5，得 D={5}；在 b 下由 3→4、4→5，得 C={4,5}。C 讀 a / b 都得 D。D 讀任何字元皆 ∅，令 X=∅。

29｜完整 DFA：每一格都交代

A={1,2}；B={3,4,5}；C={4,5}；D={5}；X=∅。B、C、D 接受，因都含 5。

DFA	a	b
→A	B	A
★B	D	C
★C	D	D
★D	X	X
X	X	X

Worklist：a 先、b 後，FIFO

取出	本輪新增	本輪後 queue
A	B	[B]
B	D, C	[D,C]
D	X	[C,X]
C	無	[X]
X	無	[]：結束

seen 存已見集合；**worklist** 存尚未處理全部字元的集合。每個新集合只加入一次；表要包含已見集合的轉移。

驗算幾個字串

輸入	走 DFA	最後
λ	A	拒
bbb	A→A→A→A	拒
a	A→B	接受
ab	A→B→C	接受
abb	A→B→C→D	接受
aaa	A→B→D→X	拒

> **∅ 不是「未算」**：沒有 NFA 路徑存活，成為 DFA 的拒絕 sink；在所有字元下仍到 ∅。原圖若省略 sink 是部分圖，完整轉移表要補上。

30｜Why correct? Why terminate?

英文說理題＋補充不變量

Initially, epsilon closure gives every NFA state reachable without input. After each symbol, move collects all symbol transitions and closure adds every epsilon–reachable state. Thus the subset always equals the possible NFA positions after that prefix. Both automata accept exactly when this subset intersects the NFA accepting set.

終止：n 個 NFA states 至多有 2^n 個子集；只處理可達集合，且每個至多一次。2^n 是**上限**，不是每題一定生成的個數。此例 5 個 NFA state → 5 個可達 DFA state。

(a|b)*abb｜構造、追蹤與合併

31｜先用一句話說出語言，再畫狀態機

Over the alphabet {a,b}, (a|b)*abb denotes all strings ending in abb.

前綴可為任意 a / b 字串（含 λ），尾端必須是 abb；最短接受字串是 **abb**，不是 ab。教材 p.41 另有 9-state NFA，**本頁只用 p.49 的編號**。

32｜原 NFA：q_0=0、F={10}

NFA	a	b	λ
→0	∅	∅	{1,7}
1	∅	∅	{2,4}
2	{3}	∅	∅
3	∅	∅	{6}
4	∅	{5}	∅
5	∅	∅	{6}
6	∅	∅	{1,7}
7	{8}	∅	∅
8	∅	{9}	∅
9	∅	{10}	∅
★10	∅	∅	∅

結構：0→7 允許零次前綴；1→2 / 4 選 a / b；3 / 5→6→1 重複；6→7 離開迴圈。7—a→8—b→9—b→10 完成尾碼。

起始閉包 → 第一個 a

```
close({0}) = {0,1,2,4,7} = A
move(A,a)  = {3,8}
close({3,8}) = {1,2,3,4,6,7,8} = B
```

3 的 λ 到 6，再到 1、7，再到 2、4。**閉包不是只走一次 λ**；要一路到固定點。

abb 的完整集合追蹤

A —a→ B —b→ D —b→ E。最後含 10，接受。NFA 的「任意前綴」與「尾碼開始」可同時存活，DFA 用集合一起保留。

33｜所有可達子集與完整 DFA

名稱	NFA state 集合
→A	{0,1,2,4,7}
B	{1,2,3,4,6,7,8}
C	{1,2,4,5,6,7}
D	{1,2,4,5,6,7,9}
★E	{1,2,4,5,6,7,10}

DFA	a	b
→A	B	C
B	B	D
C	B	C
D	B	E
★E	B	C

逐輪：A 發現 B、C；B 發現 D；C 無新集合；D 發現 E；E 無新集合。只有 E 含 10。在 Σ={a,b} 下**沒有可達 ∅**，不可硬加進可達集合清單。

字串	DFA path	結果
aabb	A→B→B→D→E	✓
babb	A→C→B→D→E	✓
abba	A→B→D→E→B	✗
ababb	A→B→D→B→D→E	✓

E 是接受、不是停止：追加 a 會回 B，不再接受；追加 b 回 C。問整串辨識，必須把輸入讀完。

34｜最小化：先接受 / 拒絕，再拆　補充

兩 states 只有對**所有後續字串**接受結果一致才可合併；不是「都拒絕」就能合併。先按 F 分組，再按各字元後繼的組別反覆細分。

輪	分組（每格表示一組）
P_0	{E} / {A,B,C,D}
P_1	{E} / {D} / {A,B,C}
P_2	{E} / {D} / {B} / {A,C}
P_3	與 P_2 相同；固定點

D 的 b 後繼接受，所以先分 D；接著 B 的 b 後繼到 D，所以分 B。A、C 兩個字元的組別始終相同，可合併。

35｜最小 DFA：用尾碼進度理解四個 states　補充，不是原投影片新增要求

最小 state	a	b	已讀字首的最長尾段 / 接受
→U={A,C}	B	U	λ：尚未匹配 abb 的非空字首
B	B	D	a
D	B	E	ab
★E	B	U	abb；接受

不能再合併：E 與其餘由 λ 區分；D 與 B / U 由 b 區分；B 與 U 由 bb 區分。這些 distinguishing suffixes 會讓其中一方接受、另一方拒絕，故至少需要四個 states。

綜合題 × 英文短答 × 最後檢查

36｜(a|(bc)*d)+：先理解「一塊」

PPT 58–60

一塊是 a，或零個以上 bc 加 d；整串由**至少一塊**串接。因此 d、a、ad、bcd、abcbcdad 接受；λ、bc、bd 拒絕。**內層 * 可零次，不代表外層 + 可零塊。**

37｜教材 NFA：start=1，F={12}

Σ={a,b,c,d}；表中 — 表示 ∅（無後繼）。

state	a	b	c	d	λ
→1	—	—	—	—	{2}
2	—	—	—	—	{3,6}
3	{4}	—	—	—	—
4	—	—	—	—	{5}
5	—	—	—	—	{2,12}
6	—	—	—	—	{7,10}
7	—	{8}	—	—	—
8	—	—	{9}	—	—
9	—	—	—	—	{7,10}
10	—	—	—	{11}	—
11	—	—	—	—	{5}
★12	—	—	—	—	—

抓結構：2 分到 a 或 (bc)*d；完成一塊到 5；5→2 可再做一塊、5→12 可結束。沒有起始→12 的零次 λ 旁路。

38｜英文短答：Definition / Difference

What is epsilon closure?

It is the set of states reachable from a given set through zero or more epsilon transitions, including the original states.

Is an NFA more powerful than a DFA?

No. Both recognize regular languages. Subset construction converts an NFA into an equivalent DFA.

What is a dead state?

It is a non-accepting state from which no accepting state is reachable. The empty subset in subset construction is a sink that loops on every input symbol.

39｜DFA 集合與完整轉移表

state	NFA states
→A	{1,2,3,6,7,10}
★B	{2,3,4,5,6,7,10,12}
C	{8}
★D	{2,3,5,6,7,10,11,12}
E	{7,9,10}
X	∅

DFA	a	b	c	d
→A	B	C	X	D
★B	B	C	X	D
C	X	X	E	X
★D	B	C	X	D
E	X	C	X	D
X	X	X	X	X

逐輪（a,b,c,d；FIFO）：A 新增 B、C、X、D；B 無新；C 新增 E；X、D、E 無新。含 12 的 B / D 才是接受。

```
abcbcdad：
A → B → C → E → C → E → D → B → D ✓
```

最短反例：bc 停 E，尚缺 d；bd 由 C 在 d 下到 X，少了 c。將路徑與「一塊」的定義互相核對。

40｜英文短答：Why / Scenario

Why convert RE to NFA and then to DFA?

NFAs are easy to build compositionally from regular expressions. A DFA then supports deterministic scanning with one next state per input symbol.

When is a subset accepting?

When it contains at least one accepting NFA state; it does not require all member states to be accepting.

Why must a scanner remember the last acceptance?

A longer candidate may fail after a shorter valid token prefix. Remembering the last accepting position allows the scanner to return the longest valid prefix.

41｜考場作答模板：把過程寫成能給分的形式

Explain：定義 → 工作 / 輸入輸出 → 小例子。**Why：**能力需求 → 比較限制 → 實作理由。**Draw AST：**寫語言假設 → 根與子樹 → 優先序 / 結合性。**Convert FA：**列 Q、Σ、start、F → close 起始 → 每字元 move 再 close → 去重 → 完整表 / 圖 → 標接受 → 測 λ、最短成功、近似失敗。

最常漏：λ≠∅；DFA 接受要讀完；NFA 接受是「存在」；closure 走到底；* 含零次；+ 看內層是否含 λ；keyword 同長才比規則；AST 不先算成常數；依題目採教材或一般語意。

來源 / 範圍：Ch03 scanning.pptx（60 slides）與六份伴讀；前兩次小考題型；AST 課本例取 Ch2 p.42。圖均重繪；C-like 練習、maximal munch、反例、最小化為補充。10 頁 A4（雙面 5 張），未假定允許頁數上限。
教材疑點已辨析：Ch3 p.47 閉包、p.51 字串 vs 語言、p.54 exponent；Ch2 p.42 右結合。Ch2 p.34 / 43 待核對仍見舊筆記。
參照：原 PPTX · 本章導讀 · Quiz 1 · Quiz 2 · flex Matching。
列印：A4 / 100% / 關閉瀏覽器自帶頁首頁尾；HTML 離線可讀。沒時間時：p.1 查名詞、p.2 按流程；CFG / AST 詳解 p.3–4；p.9 最小化為補充。